Entrega de Desenvolvimento de Operações Ágeis

Descreve cada requisito da disciplina que o projeto atende e leva, por link direto, ao arquivo ou à pasta que o comprova na branch main.

Período	Responsável	Versão	Classificação
2026	Equipe do projeto	1.0 (02/11/2026)	Entrega acadêmica

Sumário

O que há neste documento

01
Visão geral 3
O que a disciplina pede e como ler o documento

02
Requisitos mínimos 4
Pull request com aprovação, nuvem, contêineres e Kubernetes

03
Requisitos extras, README e template 5
README do projeto e template de pull request

04
Requisitos extras, pipeline e entrega 6
Pipeline de implantação e documento de entrega

05
Entregas adicionais 7
O que o projeto reúne nesta área

06
Repositórios da disciplina 8
Onde fica cada evidência

01

O projeto atende os requisitos descritos com evidência no GitHub

O que a disciplina pede e como este documento se organiza?

O que a disciplina pede

A disciplina de Desenvolvimento de Operações Ágeis pede práticas de entrega contínua, com revisão de código por pull request, infraestrutura em nuvem, contêineres e orquestração. Há requisitos mínimos, que valem a nota 7,0, e requisitos extras.

O que o projeto entregou

O projeto usa pull requests com regras de proteção da branch principal, publica a produção no Google Cloud em um cluster Kubernetes gerenciado por GitOps e empacota os serviços em imagens Docker. Os repositórios têm README, template de pull request e pipelines de build, testes e publicação de imagem. Este documento descreve cada requisito tratado e mostra onde ele está.

Como ler este documento

Cada requisito tem um identificador (M para mínimo e E para extra), o enunciado da disciplina, a descrição do que o projeto entrega e, quando há arquivo ou pasta que o comprova, um ou dois links diretos. Os links apontam para a branch main e abrem o arquivo ou a pasta que comprova o requisito. Quando o mesmo padrão existe em vários repositórios, o documento mostra um exemplo e cita os demais pelo nome.

Requisitos descritos neste documento

Identificador	Requisito	Tipo
M1	Code review e pull request com aprovação	Mínimo
M2	Cloud como infraestrutura	Mínimo
M3	Contêineres e orquestração	Mínimo
E1	README do projeto	Extra
E2	Template de pull request	Extra
E3	Pipeline de implantação	Extra
E4	Documento de entrega com link direto	Extra

02

A entrega usa pull request com aprovação, Google Cloud, Docker e Kubernetes

Como o projeto revisa o código, onde ele roda e como é empacotado?

M1 Code review e pull request com aprovação

Aplicar Code Review e Pull Request juntamente com as aprovações.

Os rulesets de proteção de branch, guardados no docs-warehouse, exigem pull request, uma aprovação, revisão do responsável pelo código e merge por squash na branch padrão, além de revisão automática do Copilot. O api-core mostra o processo no dia a dia. Ele tem template de pull request, o arquivo CODEOWNERS com o revisor responsável e o workflow pr-title-lint.yml, que valida o título do PR a cada abertura.

- pasta github/rulesets em docs-warehouse
- CODEOWNERS em api-core

M2 Cloud como infraestrutura

Utilizar a Cloud como infraestrutura para hospedar o projeto.

A produção roda no Google Cloud. O Terraform do infra-platform descreve a rede, o cluster GKE, o NAT e o Argo CD, e o infra-gitops mantém o estado desejado do cluster.

- modules/cluster/main.tf em infra-platform
- README.md em infra-gitops

M3 Contêineres e orquestração

Utilizar tecnologia de container e orquestração (Docker e Kubernetes).

Os serviços são empacotados em imagens Docker, e mais de 15 repositórios têm Dockerfile, entre eles api-core, web-app e ai-assistant. A orquestração usa Kubernetes. O infra-gitops guarda os manifests de cada serviço (Deployment, Service e Ingress), uma aplicação do Argo CD por serviço e a aplicação raiz que as reúne.

- Dockerfile em api-core
- pasta services em infra-gitops

03

Os repositórios têm README completo e abrem pull requests com um template

Como o projeto documenta o código e padroniza as mudanças?

E1 README do projeto

Criar o README do projeto, completo.

O README do api-core aponta para a documentação em docs e flows, que inclui ARCHITECTURE.md, RUNNING.md e DEPLOY.md, com a arquitetura, a forma de rodar e a forma de implantar o serviço. O mesmo padrão de README aparece também em web-app, ai-assistant e infra-platform.

- README.md em api-core
- ARCHITECTURE.md em api-core

E2 Template de pull request

Definir Template de Pull Request e aplicar durante o desenvolvimento.

O template do api-core abre o pull request com as seções Objetivo, Alterações, Endpoints afetados, Banco de Dados, Como testar e Evidências. Cada pull request abre com esse roteiro. O arquivo pull_request_template.md existe também em web-app, ai-assistant, infra-platform e infra-gitops.

- pull_request_template.md em api-core

04

O pipeline publica e implanta os serviços, e este documento cumpre a entrega

Como o código chega ao ambiente e como a disciplina é comprovada?

E3 Pipeline de implantação

Elaborar pipeline para implantação (GitHub Actions ou Jenkins).

O api-core roda os testes com cobertura a cada pull request e a cada push na main pelo workflow ci.yml, e publica a imagem do serviço pelo release.yml. Ele ainda tem versionamento e changelog automáticos (release-please.yml) e análise de qualidade (sonarqube.yml). Na implantação, o Argo CD sincroniza o cluster a partir da main do infra-gitops. O web-app publica o Storybook no GitHub Pages pelo workflow preview.yml.

- ci.yml em api-core
- pasta apps em infra-gitops

E4 Documento de entrega com link direto

Entrega, com documento de entregas descrevendo cada requisito, com link direto para o requisito.

Este documento é a entrega pedida. Ele descreve cada requisito tratado e traz, para cada um, um link direto para o arquivo ou a pasta que o comprova na branch main.

05
Entregas adicionais

O que o projeto reúne nesta disciplina?

A infraestrutura do projeto é descrita como código com Terraform em infra-platform, que cria a rede, o cluster GKE, o Argo CD, o Kong, o cert-manager, o DNS e o acesso na Cloudflare e as leituras do cofre de segredos Infisical, enquanto infra-gitops mantém o estado desejado do cluster em Applications do Argo CD. O cluster é efêmero, ligado e desligado por scripts, e o QA roda em um serviço gratuito mantido ativo por infra-keepalive. A observabilidade usa o coletor OpenTelemetry de infra-otel-collector, os painéis e as quinze regras de alerta de infra-observability e o worker-alerts, que transforma cada alerta em um incidente no Jira, e infra-stress traz testes de carga em k6. O infra-gateway configura o Kong, o worker-failover cuida da entrada de borda, o generic-template padroniza novos repositórios e o repositório .github disponibiliza workflows reutilizáveis de integração contínua e de release.

Links de exemplo, infra-observability na branch main e infra-keepalive na branch main.

06

A evidência da disciplina está distribuída em quinze repositórios

Qual repositório guarda cada parte da entrega?

Repositório	Papel na disciplina	Raiz
api-core	Serviço principal do projeto. Traz template de pull request, responsável pela revisão, workflows de build, testes e publicação de imagem, Dockerfile e README.	api-core na branch main
web-app	Interface web. Traz Dockerfile, README e o workflow que publica o Storybook no GitHub Pages.	web-app na branch main
ai-assistant	Agente de IA do projeto. Traz Dockerfile e README.	ai-assistant na branch main
infra-platform	Infraestrutura em Terraform no Google Cloud, com rede, cluster GKE e Argo CD. Traz README e template de pull request.	infra-platform na branch main
infra-gitops	Estado desejado do cluster. Traz os manifests Kubernetes de cada serviço e as aplicações do Argo CD.	infra-gitops na branch main
docs-warehouse	Documentação de apoio e rulesets de proteção de branch do GitHub.	docs-warehouse na branch main
infra-otel-collector	Coletor OpenTelemetry que reúne registros, métricas e rastros dos serviços.	infra-otel-collector na branch main
infra-observability	Painéis do Grafana e regras de alerta da plataforma.	infra-observability na branch main
worker-alerts	Worker que transforma os alertas do monitoramento em incidentes no Jira.	worker-alerts na branch main
infra-stress	Testes de carga em k6 da aplicação web e da entrada de borda.	infra-stress na branch main
infra-keepalive	Rotina agendada que mantém ativos os serviços do QA.	infra-keepalive na branch main
infra-gateway	Configuração do gateway Kong para o QA e para o cluster.	infra-gateway na branch main
worker-failover	Cloudflare Worker de entrada de borda, com página temporária e telemetria.	worker-failover na branch main

generic-template	Modelo para novos repositórios, com README, coleção Bruno, integração contínua e pull request.	generic-template na branch main
.github	Workflows reutilizáveis de integração contínua e de release, disponíveis aos repositórios.	.github na branch main



Entrega acadêmica.